

Connectionist Computing Multi-Layer Perceptron Assignment

Roniya Pinto (22385491)

Abstract

The goal of this project was to build a multi-layer perceptron (MLP) in any programming language without using any neural network, connectionist, or machine learning libraries. The software is capable of creating an MLP with a configurable number of inputs, outputs and hidden units in a single hidden layer. The weights are initialized to small random values and the network can predict outputs for a given input and learn via backpropagation. The MLP was tested on three examples: XOR, sine regression and letter recognition. This report briefly outlines the building of the MLP and focuses on the experiments and results obtained.

1 Introduction

This project implements and evaluates multi-layer perceptrons (MLPs) for classification and regression tasks entirely from scratch. The objectives are:

- Implement a fully configurable MLP with one hidden layer.
- Initialize network weights to small random values.
- Compute predictions for given input vectors.
- Train the network using backpropagation.
- Test the implementation on three representative datasets:
 1. XOR: simple binary classification.
 2. Sine regression: nonlinear function approximation.
 3. Letter recognition: multi-class real world classification.

The report emphasizes the experimental setup, training procedures, evaluation metrics and results for each task.

2 Coding Choices

The multi-layer perceptron (MLP) implemented for this project was written entirely in Python, chosen due to familiarity and its concise syntax for numerical computation with `numpy`. The network consists of a single hidden layer with a configurable number of neurons, which can employ either the `tanh` or `sigmoid` activation function depending on the task. The output layer is also configurable, supporting `sigmoid` for binary classification, `softmax` for multi-class classification and `linear` for regression tasks. Bias terms are included in both hidden and output layers to provide additional flexibility in learning and all weights are initialized randomly within the range $[-0.5, 0.5]$ to break symmetry and facilitate effective training.

The forward pass of the network first concatenates the bias term to the input, computes the hidden layer activations and then computes the output by applying the chosen activation function. In the case of softmax, a numerical stabilization term is subtracted prior to exponentiation to prevent overflow.

Training is performed using standard backpropagation. For each sample, a forward pass is used to compute the outputs, followed by computation of the error deltas at the output and hidden layers. The derivatives of the activation functions, specifically `dtanh` and `dsigmoid`, are employed to propagate the error gradients backward. There is no separate or explicit weight update function; instead the weight matrices are updated directly within the backward pass after processing each sample. This means that weights are updated once per training example, which is effectively stochastic gradient descent. The updates use the learning rate and the outer product of the corresponding deltas with the input or hidden activations. Updating weights per example allows the network to respond immediately to each input, helps avoid being stuck in local minima, and introduces small stochasticity that can aid generalization.

The network tracks the mean squared error and cross-entropy, depending on the task, over each epoch and these values are recorded to a file to monitor convergence. Predictions on new data are obtained simply by performing a forward pass without updating weights. The implementation allows full configurability of the number of inputs, hidden units, outputs, output activation type, learning rate and number of epochs. This flexibility enables the MLP to be tested across diverse tasks such as XOR classification, sine regression and letter recognition.

Hyperparameters Used in All Experiments

Experiment	Inputs	Hidden	Outputs	LR	Epochs	Output
XOR Classification	2	4	1	0.5	5000	Sigmoid
Sine Regression	4	10	1	0.1	1000	Linear
Letter Recognition	16	30	26	0.01	1000	Softmax

Table 1: Hyperparameters used across all MLP experiments.

3 Experiments and Results

3.1 XOR Classification Experiment

- Architecture: 2 inputs, 4 hidden neurons, 1 output (sigmoid).
- Training: 5000 epochs, learning rate 0.5.
- Metrics: final predictions, training error curve.

The XOR experiment demonstrates the ability of the implemented multi-layer perceptron (MLP) to learn a non-linearly separable binary classification problem. The XOR dataset consists of four input vectors with two features each, representing all possible combinations of binary inputs: $[0, 0]$, $[0, 1]$, $[1, 0]$ and $[1, 1]$. The corresponding targets are $[0]$, $[1]$, $[1]$ and $[0]$, reflecting the XOR truth table.

For this experiment, an MLP with two input neurons, four hidden neurons and one output neuron was configured. The hidden layer uses the `tanh` activation function, while the output layer uses the `sigmoid` activation function to produce outputs in the range $[0, 1]$. The weights were initialized randomly in the range $[-0.5, 0.5]$ and a fixed random seed was used to ensure reproducibility.

The network was trained using stochastic gradient descent with backpropagation over 5000 epochs and a learning rate of 0.5. A learning rate of 0.5 was chosen because the XOR problem is small and low dimensional, allowing the network to take large gradient steps without instability, which accelerates convergence. During each epoch, the order of the four training samples was randomly permuted to avoid bias in weight updates. After each forward and backward pass, the error was accumulated and the mean squared error (MSE) was recorded. The training process and MSE at regular intervals were logged to `"xor_training_error.txt"` for analysis.

After training, the network was tested on all four inputs and the predicted outputs were compared with the target values.

3.2 XOR Experiment Results

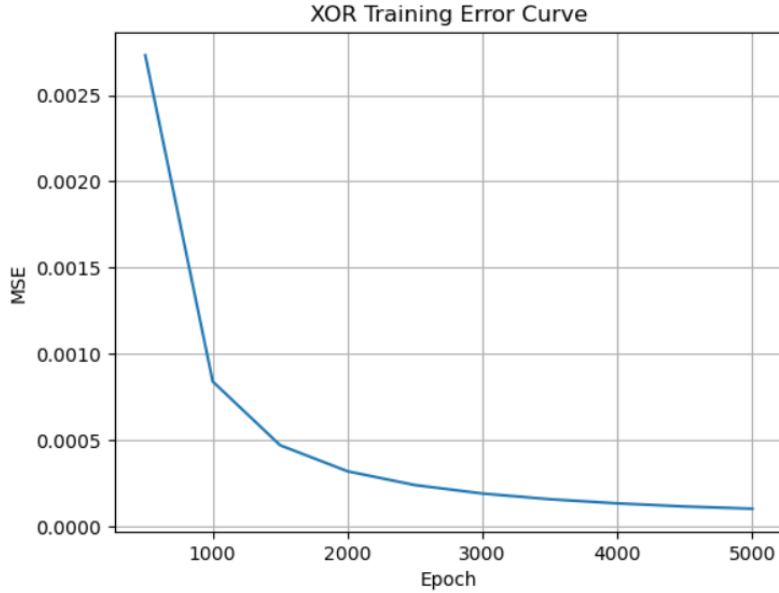


Figure 1: Training error curve for the XOR experiment. Mean squared error decreases over 5000 epochs, showing convergence of the MLP.

The XOR Training Error Curve illustrates how the neural network learns to solve the non-linearly separable XOR problem over 5000 training epochs. Initially, the MSE is high, but it drops sharply within the first 1000 epochs, indicating rapid learning of the basic structure. Between 1000 and 3000 epochs, the error declines more gradually as the network refines its internal representations and adjusts weights to capture the non-linear patterns inherent in XOR. From 3000 to 5000 epochs, the curve flattens, suggesting convergence and diminishing returns from further training. The smooth and consistent decline in error reflects stable optimization. The absence of oscillations or spikes indicates effective learning without instability and the lack of a rising error trend suggests no overfitting, though validation metrics would be required for confirmation.

After training, the network was tested on all four XOR inputs. The final predictions are summarized in Table 2.

Input	Target	Output	Correct
[0, 0]	0	0.0100	1
[0, 1]	1	0.9893	1
[1, 0]	1	0.9922	1
[1, 1]	0	0.0112	1

Table 2: Final predictions of the MLP on the XOR dataset. Correct=1 indicates the output is closest to the target.

The network successfully learned the XOR function. Numerically, the network’s outputs are very close to the target values. For the input [0, 0], the predicted output is 0.0100,

effectively zero. For $[0, 1]$ and $[1, 0]$, the outputs are 0.9893 and 0.9922, respectively, nearly one. For $[1, 1]$, the output is 0.0112, again effectively zero. These results show that the network not only classifies the inputs correctly when rounded to binary values but also produces outputs with high confidence, demonstrating that it has learned a strong internal representation of the XOR function. Overall, the results demonstrate that the chosen architecture, activation functions and learning rate were sufficient for the network to reliably solve the XOR problem.

3.3 Sine Regression Experiment

- Dataset: 500 samples, each with 4 input features scaled to the range $[-1, 1]$, with targets computed as $Y = \sin(x_0 - x_1 + x_2 - x_3)$.
- Architecture: 4 inputs, 10 hidden neurons, 1 output (linear).
- Training: 1000 epochs, learning rate 0.1.
- Metrics: train/test MSE, mean absolute error, R^2 .

The sine regression experiment evaluates the ability of the implemented MLP to approximate a non-linear continuous function. A dataset of 500 samples was generated, each consisting of four input features sampled uniformly from the interval $[-1, 1]$. The target output for each sample was computed as $Y = \sin(X_0 - X_1 + X_2 - X_3)$, creating a non-linear relationship between the inputs and output that the network must learn.

For this task, the MLP was configured with four input neurons, ten hidden neurons and one output neuron. The hidden layer used the `tanh` activation function, while the output layer used a linear activation function to produce continuous values suitable for regression. The network was trained using stochastic gradient descent with backpropagation over 1000 epochs and a learning rate of 0.1. A learning rate of 0.1 was selected after experimentation to balance stability and speed, since the continuous regression task requires smoother weight updates to accurately approximate the sine function. The first 400 samples were used for training, while the remaining 100 samples were held out for testing.

During training, the mean squared error (MSE) was computed for each epoch and logged to `"sin_training_error.txt"` to monitor convergence. Predictions were generated for both the training and test sets.

3.4 Sine Regression Results

After training the MLP on the sine regression dataset, the performance metrics on the training and test sets are summarized in Table 3.

Metric	Value
Training MSE	1.832×10^{-5}
Test MSE	2.843×10^{-5}
Mean Absolute Error (MAE)	0.00378
R^2 Score	0.99994

Table 3: Performance metrics of the MLP on the sine regression dataset.

The final error on the training set is extremely low and the error on the test set is similarly minimal, indicating that the network generalizes well and has not overfitted. The high R^2 score confirms that the model captures nearly all of the variance in the target data. These results demonstrate that the network has learned the sine function satisfactorily.

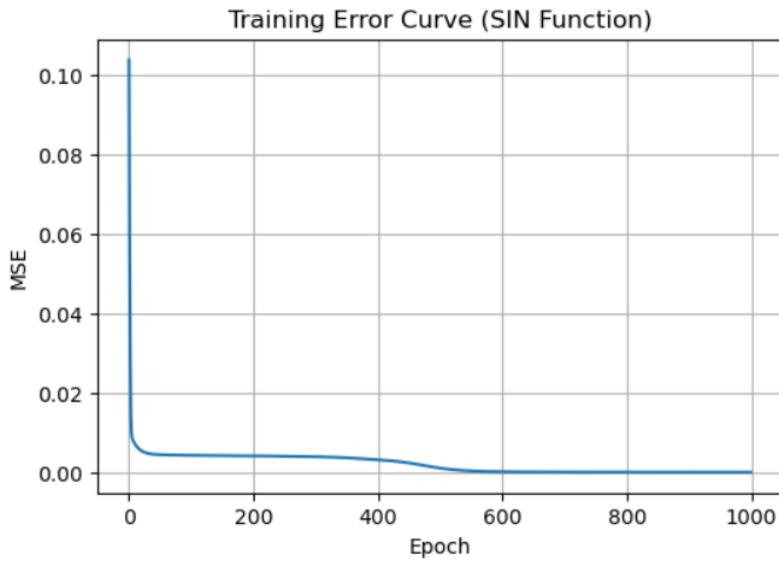


Figure 2: Training error curve for the sine regression experiment. The MSE drops rapidly in early epochs and flattens as the network converges.

The “Training Error Curve” illustrates the model’s learning progression over 1000 epochs. The error decreases sharply within the first 200 epochs, reflecting rapid learning of the fundamental sine structure and then gradually flattens, indicating convergence and minimal residual error. The smooth decline demonstrates stable optimization and appropriate hyperparameters.

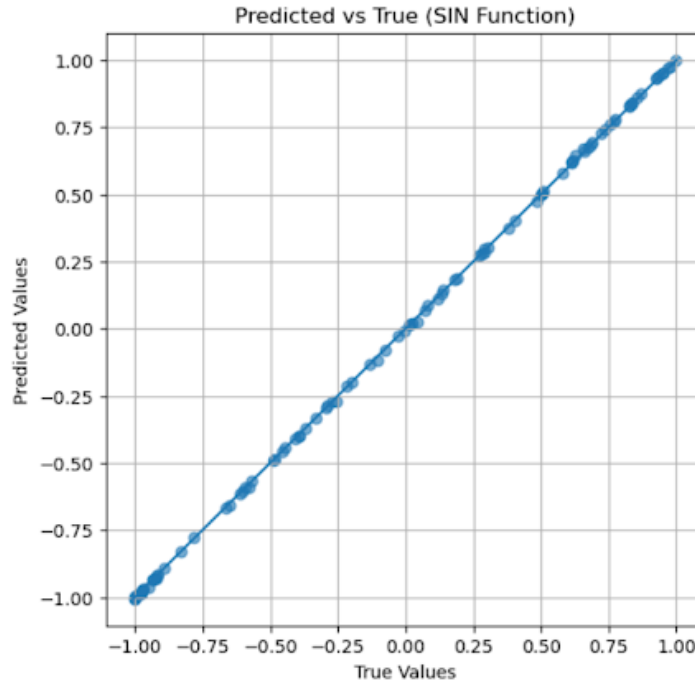


Figure 3: Predicted vs True values for the sine function. The tight clustering along the diagonal indicates high predictive accuracy.

The “Predicted vs True” scatter plot shows a strong correlation between predicted and actual values. The points cluster tightly around the diagonal from bottom left to top right, indicating that the network accurately captures both the amplitude and phase of the sine function. The uniform coverage across the input range further confirms generalization.

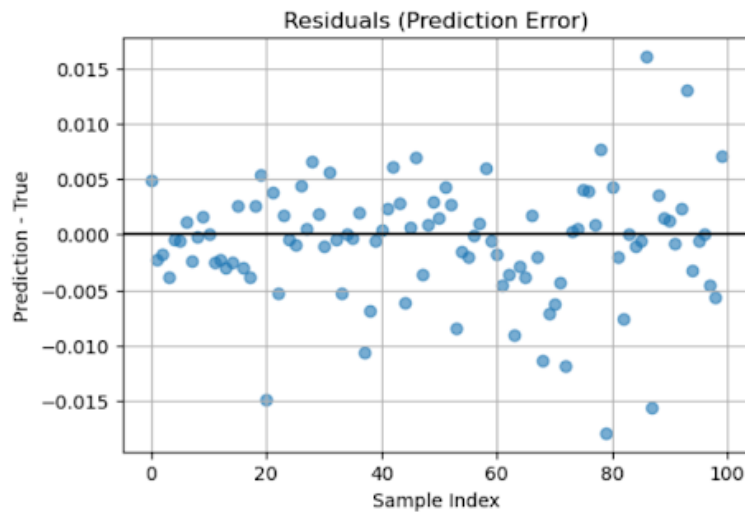


Figure 4: Residuals (prediction errors) for the sine regression model. Errors are tightly clustered around zero, indicating minimal and randomly distributed prediction error.

The “Residuals” plot highlights the consistency of the network’s predictions. Residuals are tightly clustered around zero with no discernible patterns, suggesting that the errors

are random rather than systematic. The narrow error range confirms high precision across the dataset and supports the conclusion that the network generalizes well.

Overall, the combination of extremely low training and test errors, a near perfect R^2 score and visual confirmation from the training curve, predicted vs true and residuals plot demonstrates that the network has successfully learned the underlying structure of the sine function and generalizes effectively to unseen data.

3.5 Letter Recognition Experiment

- Dataset: UCI Letter Recognition (16 attributes, 26 classes).
- Architecture: 16 inputs, 30 hidden neurons, 26 outputs (softmax).
- Training: 1500 epochs, learning rate 0.01.
- Metrics: train/test accuracy, confusion matrix.

The letter recognition experiment evaluates the performance of the implemented MLP on a multi-class classification task. The dataset consists of samples representing uppercase letters A–Z, with each sample described by 16 numerical features extracted from images of handwritten letters. To prepare the data for the MLP, each letter was converted into a one-hot encoded vector of length 26, where the position corresponding to the letter is set to 1 and all other positions are set to 0. The feature values were normalized by dividing by 15 to scale inputs to the range $[0, 1]$.

The dataset was randomly shuffled and split into training and test sets, with 80% of samples used for training and 20% reserved for testing. The network architecture consists of 16 input neurons (corresponding to the features), a single hidden layer with 30 neurons employing the `tanh` activation function and an output layer with 26 neurons using the `softmax` activation function to produce a probability distribution over all letter classes.

Training was performed using stochastic gradient descent with backpropagation over 1000 epochs and a learning rate of 0.01. A learning rate of 0.01 was used because the dataset is high dimensional and multi-class and a smaller rate prevents divergence and ensures stable training when optimizing the softmax cross-entropy loss. During training, the cross-entropy loss between predicted and target outputs was computed for each epoch and recorded to "`letter_training_error.txt`" for monitoring convergence. After training, predictions for the test set are obtained by performing a forward pass through the network and the predicted class for each sample is determined by selecting the output neuron with the highest activation.

3.6 Letter Recognition Results

The performance of the MLP on the letter recognition dataset is summarized in Table 4. Overall, the network achieved a high accuracy of 90.5%, with macro averaged precision,

recall and F1 scores all above 0.90, indicating consistently strong performance across classes.

Metric	Value
Overall Accuracy	0.905
Macro Precision	0.9061
Macro Recall	0.9043
Macro F1 Score	0.9040

Table 4: Overall performance metrics for the letter recognition MLP.

Table 5 provides a detailed breakdown of precision, recall, F1 score and support (number of test samples per letter) per letter. The network performs very well for most letters, with precision and recall values generally above 0.85. Letters such as A, I and P achieve particularly high precision and recall, while some letters like H and D show slightly lower recall, indicating occasional misclassifications. Overall, the model demonstrates strong multi-class classification capability.

Letter	Precision	Recall	F1	Support
A	0.9659	0.9659	0.9659	176
B	0.8865	0.8865	0.8865	141
C	0.9510	0.8662	0.9067	157
D	0.9441	0.7941	0.8626	170
E	0.8681	0.8446	0.8562	148
F	0.9209	0.8477	0.8828	151
G	0.8636	0.9236	0.8926	144
H	0.7933	0.8380	0.8151	142
I	0.9405	0.9518	0.9461	166
J	0.9790	0.8750	0.9241	160
K	0.8235	0.8936	0.8571	141
L	0.9273	0.9563	0.9415	160
M	0.9341	0.9630	0.9483	162
N	0.9600	0.8571	0.9057	140
O	0.8627	0.9041	0.8829	146
P	0.9382	0.9709	0.9543	172
Q	0.8490	0.9760	0.9081	167
R	0.8363	0.8563	0.8462	167
S	0.9178	0.8874	0.9024	151
T	0.8675	0.9172	0.8916	157
U	0.9557	0.9207	0.9379	164
V	0.9348	0.9485	0.9416	136
W	0.9716	0.9320	0.9514	147
X	0.8986	0.8926	0.8956	149
Y	0.9301	0.8926	0.9110	149
Z	0.8387	0.9489	0.8904	137

Table 5: Per letter performance metrics for the letter recognition MLP.

These results show that the MLP effectively captures the distinguishing features of handwritten letters, providing high overall accuracy and balanced performance across classes. Letters with slightly lower recall may benefit from additional training data or further tuning of the network architecture.

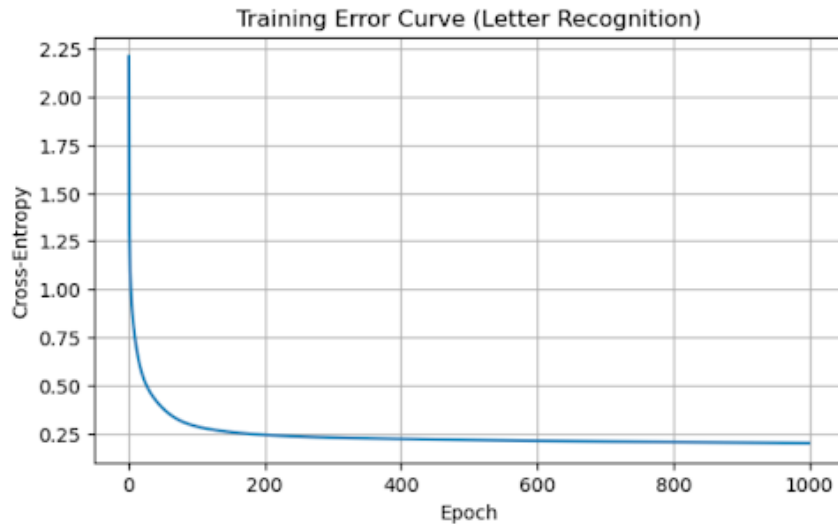


Figure 5: Training error curve for the letter recognition experiment.

The training error curve illustrates the learning dynamics of the multilayer perceptron. At the beginning of training, the error exceeds 2.25, reflecting the network's initially uninformed predictions. Over the first few hundred epochs, the loss decreases sharply as the model rapidly identifies useful structure within the 16 input features. After approximately 200 epochs, the curve begins to flatten and stabilizes just above 0.25, indicating convergence. This smooth decline signals effective optimization and confirms that the architecture and learning rate are well suited to the complexity of the 26 class classification task.

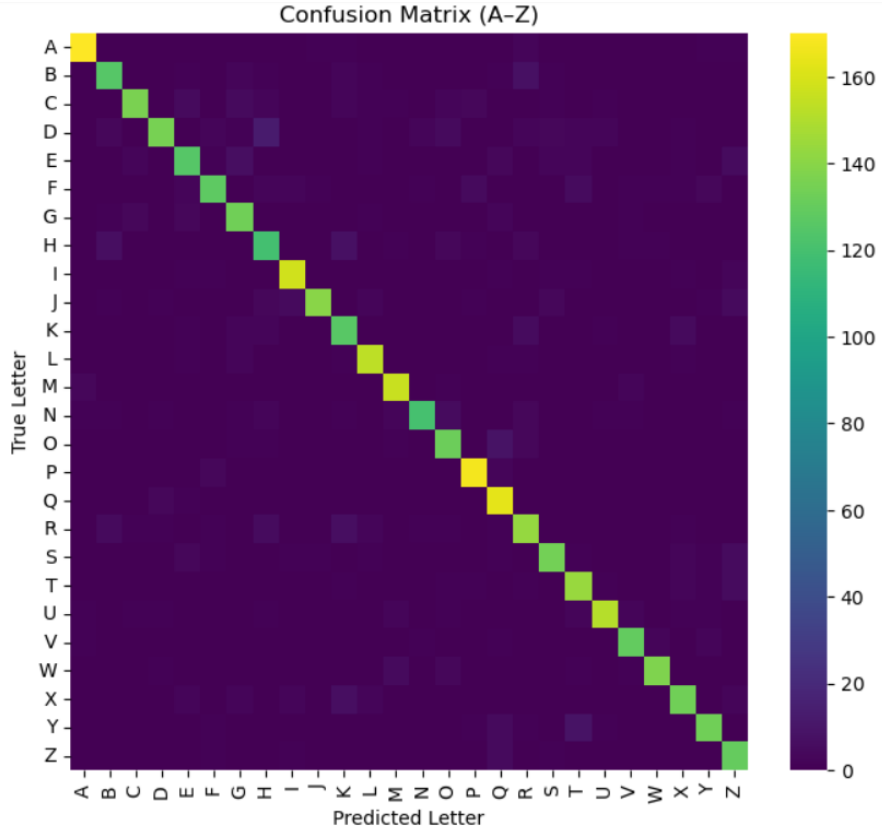


Figure 6: Confusion matrix for A-Z letter classification.

The confusion matrix provides a detailed breakdown of the model’s classification performance for each letter. A strong diagonal band from top left to bottom right signifies that most predictions match the true labels, which aligns with the high overall accuracy achieved. Many diagonal entries exceed 150 correct classifications, demonstrating strong performance across most classes. Off diagonal entries indicating misclassifications are relatively sparse and generally low in magnitude. Overall, the matrix reveals a well generalized model.

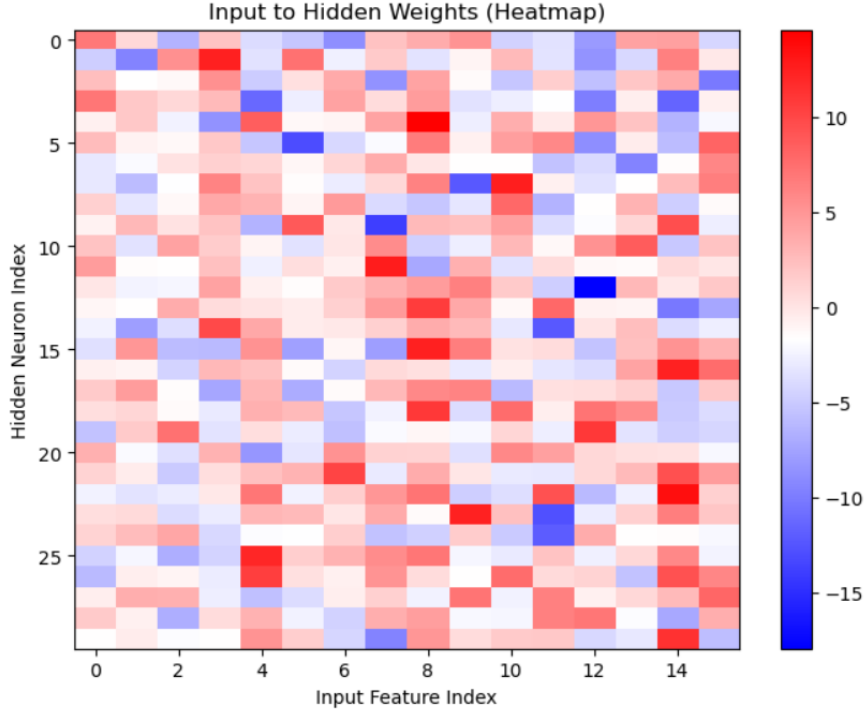


Figure 7: Input to hidden layer weight heatmap for the letter recognition MLP.

The weight heatmap visualizes the learned input to hidden layer weights and provides insight into how the network encodes the 16 numerical features. The high contrast pattern of positive and negative weights indicates that the hidden neurons have learned diverse and specialized feature detectors. Several rows show strong, localized weight activations on specific features, suggesting that individual neurons focus on particular geometric characteristics of the letters. No input feature appears consistently inactive, meaning that all features contribute meaningfully to classification. Complementary patterns across some neurons where one neuron emphasizes features that another suppresses, reflect the network’s ability to learn contrasting, discriminative representations. The absence of degenerate patterns such as uniform rows, zero weights, or repeated structures indicates stable optimization and confirms that the network successfully discovered a rich internal representation capable of supporting high accuracy letter classification.

4 Conclusions

Completing this assignment and implementing a multi-layer perceptron entirely from scratch significantly deepened my understanding of how neural networks function beyond the abstractions provided by high level machine learning libraries. Writing every component manually including the forward pass, activation functions and backpropagation algorithm forced me to understand how information and gradients flow through the network and how the mathematical operations translate into parameter updates. Concepts that often feel abstract in theory, such as the chain rule in backpropagation or the role of activation functions in enabling non-linearity, became much clearer once I had to implement them myself and debug their behaviour in practice.

One notable challenge I faced was designing a single MLP class that could be reused for all three experiments, which had very different requirements: binary classification (XOR), regression (sine function) and multi-class classification (letter recognition). I overcame this by introducing conditional logic and making the output type a configurable parameter, allowing the same class to switch between sigmoid, linear and softmax outputs as needed. This approach not only simplified the code but also reinforced my understanding of how network outputs, loss functions and activation functions are intertwined.

Through the experiments, I also gained a strong appreciation for how different architectural and hyperparameter choices influence learning. The XOR problem, for instance, demonstrated the necessity of a hidden layer when dealing with non-linearly separable tasks. The sine regression experiment showed how sensitive training can be to the learning rate: using a learning rate of 0.5 caused the model to diverge, while lowering it to 0.1 produced smooth, stable convergence. This helped me understand how the scale of the gradients and the complexity of the target function influence the selection of hyperparameters. Similarly, the letter recognition experiment illustrated how higher dimensional and more complex classification tasks require a larger hidden layer to capture meaningful internal representations. Training with only a few hidden units resulted in underfitting, whereas expanding the hidden layer to 30 neurons allowed the network to extract richer features and achieve high accuracy across 26 classes.

Honestly in terms of hyperparameter choice I didn't face too much difficulty. The initial values were chosen with an educated guess and then were tweaked based on performance. There was not enough tweaking done to provide sufficient meaningful data that could be presented in a graph.

Overall, building the MLP from scratch helped me see many of the processes that occur behind the scenes in modern neural network frameworks. I now have a much clearer understanding of weight initialization, error metrics, gradient descent and the practical challenges of training neural networks. More importantly, this assignment demonstrated how subtle design choices; such as activation types, network depth and learning rates directly shape performance.